CLAUDE CERTIFIED ARCHITECT — PROFESSIONAL

Domain 7: Developer Productivity & Operational Enablement

Exam objectives covered

1. Configure Claude Code and related tooling across organisation, project, and user levels — including managed settings, permissions, MCP servers, hooks, and shared project instructions — to support consistent, secure team environments.
2. Apply Claude Code commands, skills, and automation patterns to improve shared development workflows, including testing, code review, CI/CD integration, and measurable productivity gains across a team.
3. Troubleshoot Claude Code configuration, MCP server failures, hooks, permissions, context issues, and tool execution failures, and monitor usage and operational behaviour to resolve developer-facing problems.

Q1 **A global retailer's platform team wants to guarantee that no Claude Code user anywhere in the company can ever disable a specific security-scanning hook, regardless of their local preferences. Which configuration layer must enforce this?**

A User-level settings

B Project-level settings committed to a single repo

C **Managed settings deployed organisation-wide, which take precedence over all other layers**

D A note in CLAUDE.md

CORRECT ANSWER **C**

WHY Only managed settings sit above project, local, and user settings in the precedence hierarchy, making them the sole mechanism that guarantees a policy (like a mandatory hook) cannot be disabled downstream. User settings, project settings, and CLAUDE.md are all either overridable or non-enforcing.

Q2 An insurance company wants new engineers to be productive on day one by having consistent coding conventions, directory context, and a reusable "scaffold new service" command available automatically in every repo they clone. What combination achieves this?

- **A** **A CLAUDE.md file for persistent project context plus a shared custom command distributed via project or managed configuration**
- **B** Verbal onboarding sessions only, with no files
- **C** A single managed settings permission rule with no other configuration
- **D** Requiring new engineers to write their own CLAUDE.md from scratch

CORRECT ANSWER **A**

WHY Combining CLAUDE.md (persistent shared context) with a distributed reusable command gives every new engineer the same consistent experience automatically, which is the intended design pattern for shared workflows. Verbal-only onboarding or a bare permission rule don't convey workflow context, and asking each engineer to author their own instructions defeats the purpose of shared configuration.

Q3 A developer reports that a previously working MCP-based tool now returns "server unavailable" errors. What is the most systematic first diagnostic step?

- **A** Reinstall Claude Code from scratch
- **B** **Investigate the MCP server's own health, logs, authentication, and recent changes, since it runs as a separate external process**
- **C** Assume the Claude model itself is broken
- **D** Disable all permissions to see if that fixes it

CORRECT ANSWER **B**

WHY Because MCP servers are external integrations, connectivity/availability failures are most often rooted in the server's own health, auth, or configuration — checking there first is the systematic approach rather than reinstalling the client or blaming the model. Disabling permissions addresses access control, not server availability.

Q4 **A fintech company wants its CI pipeline to use Claude Code to automatically triage failing builds and open a draft PR with a proposed fix, but never push directly to a protected branch. How should this be configured?**

A Give the CI automation unrestricted repository admin access for simplicity

B Scope the CI service identity's credentials and permissions narrowly (draft PR/branch creation only, no push to protected branches), enforced through permission rules

C Run the automation with no credentials at all

D Trust the model's judgment with no technical restriction

CORRECT ANSWER B

WHY Controlled AI-assisted CI/CD automation requires least-privilege, scoped credentials combined with explicit permission rules enforcing the intended boundary — this is a deterministic technical control, not a trust-based one. Unrestricted admin access or relying purely on model judgment both violate least-privilege principles, and zero credentials would make the automation non-functional.

Q5 **A team lead wants proof that adopting a shared library of Claude Code Skills actually improved delivery, not just activity. Which measurement approach is most credible?**

A Counting how many times "Claude" is mentioned in Slack

B Comparing outcome metrics (PR cycle time, review turnaround, defect escape rate) before and after adoption, correlated with actual skill usage data

C Asking one enthusiastic developer for their opinion

D Counting total lines of code generated regardless of quality

CORRECT ANSWER B

WHY Correlating adoption data with outcome-oriented engineering metrics (cycle time, review turnaround, defects) provides defensible evidence of real productivity impact, unlike vanity metrics such as mention counts or raw code volume. A single anecdote also lacks the rigor needed for an organisational conclusion.

Q6 A platform engineer is auditing why an engineer's Claude Code session has broader file-write access than the rest of the team despite identical managed settings being deployed to everyone. What should be checked first?

- **A** **Whether local or user-level settings on that engineer's machine loosen a permission that managed settings didn't explicitly lock down**
- **B** Whether the engineer's monitor resolution is different
- **C** Whether the engineer uses a different code editor theme
- **D** Whether the repository has more commits than others

CORRECT ANSWER **A**

WHY When managed settings don't explicitly restrict a permission, lower-precedence local or user settings can still broaden access, so checking those layers is the correct first step in explaining an unexpected permission discrepancy. Hardware, editor themes, and commit counts have no bearing on the settings hierarchy.

Q7 A healthcare software company configured a hook intended to block any file write containing a detected secret, but a recent write containing an API key slipped through unblocked. What is the most likely root cause to investigate?

- **A** **The hook's exit code or output format doesn't match what Claude Code interprets as a block signal, so the deny wasn't actually honored**
- **B** The repository has too many branches
- **C** The developer's terminal color scheme
- **D** CLAUDE.md wasn't updated with a note about secrets

CORRECT ANSWER **A**

WHY Hooks communicate block/allow decisions via specific exit codes or output contracts; if the script's signal doesn't conform to what Claude Code expects, the intended block silently fails to take effect, which is the key thing to verify. CLAUDE.md is advisory text and wouldn't itself enforce a block even if updated, and the other options are irrelevant to hook execution.

Q8 **A manufacturing company's engineering org runs Claude Code across 50 teams and wants a single ticketing-system MCP integration available everywhere without each team configuring it from scratch, while still letting individual teams add their own specialized MCP servers. What is the best structure?**

A Configure the ticketing MCP server independently in every team's local settings

B **Define the shared ticketing MCP server at the managed/organisation level as a baseline, and allow project-level settings to add team-specific MCP servers within what managed settings permit**

C Disallow any project-level MCP servers to keep things simple

D Require a company-wide vote before any team can use MCP

CORRECT ANSWER **B**

WHY This layered approach matches the settings hierarchy's purpose: a managed baseline ensures consistent organisation-wide distribution of the shared tool, while project-level settings retain flexibility for team-specific extensions. Independent per-team configuration doesn't scale, and disallowing project-level additions removes legitimate flexibility.

Q9 **After a managed settings update added a new deny rule, several unrelated teams report that previously working Claude Code workflows suddenly broke. What should the platform team do first?**

A Tell each affected team to quietly work around the restriction in their own project settings

B **Review the newly added deny rule for unintended overly broad scope that may be affecting more tools or paths than intended, and correct or roll it back**

C Conclude the teams are misusing the tool and take no further action

D Remove managed settings entirely to avoid future issues

CORRECT ANSWER B

WHY Since the breakage coincided with a specific managed settings change and spans multiple unrelated teams, the most likely explanation is an overly broad new rule, making targeted review/correction the right first step. Letting teams individually bypass org policy undermines managed settings' purpose, and removing managed settings entirely abandons an essential governance control.

Q10 A developer in a long-running Claude Code session finds that instructions given at the start of the conversation are no longer being followed, even though nothing in the configuration changed. What is the most likely explanation and appropriate remedy?

A **The session has hit context pressure, causing earlier instructions to be truncated or deprioritized; the remedy is to reset/summarize context or reinforce key instructions via persistent CLAUDE.md content rather than relying on conversational memory alone**

B The installation is corrupted and must be reinstalled

C An MCP server outage is responsible

D A permission rule is silently blocking instruction-following

CORRECT ANSWER **A**

WHY Long sessions can exhaust usable context, causing earlier instructions to fade from effective attention; the correct fix is managing context (reset/summarize) and anchoring durable guidance in CLAUDE.md so it persists reliably. Reinstalling software, MCP outages, and permission rules would produce distinctly different symptoms (crashes, tool errors, access denials) rather than a gradual drift in instruction adherence.

Q11 **A regulated enterprise wants to centrally govern which Claude Code plugins (bundles of skills, commands, subagents, hooks, and MCP servers) developers can install, restricting installation to a single internally vetted marketplace and blocking all public ones — and also wants session startup to fail rather than silently use stale policy if the managed-settings server is unreachable. Which pair of managed-settings capabilities together satisfies both requirements?**

A. A CLAUDE.md note about approved plugins, plus a longer session timeout

B. **Marketplace allow/deny controls (e.g., `strictKnownMarketplaces` / `blockedMarketplaces`) to govern plugin sources, plus `forceRemoteSettingsRefresh` to fail closed when current org policy cannot be fetched**

C. Giving every developer admin rights to self-manage plugin installation

D. Disabling the Skills feature organisation-wide so no plugins can ever be used

CORRECT ANSWER B

WHY Marketplace allow/deny settings constrain which plugin sources are reachable at all, while `forceRemoteSettingsRefresh` ensures a session won't start under potentially outdated cached policy if the managed-settings server can't be reached — together these are the enforced, technical controls for this governance requirement. CLAUDE.md notes are advisory, admin self-management reintroduces the ungoverned risk, and disabling Skills entirely discards legitimate productivity capability instead of scoping the real risk.

Q12 A team wants Claude Code to automatically review every pull request in CI without a human at the keyboard, but is also worried that the same session that authored a change could review its own work and miss the same blind spots, and separately wants to ensure that if CI ever checks out an untrusted third-party contribution, that repo's own hooks or MCP servers can't silently auto-execute. Which combination of practices addresses both concerns?

A **Run the review as an independent headless-mode step (e.g., `claude -p` with JSON output and a scoped tool allowlist) separate from the authoring session, and use bare mode to suppress auto-discovery of repo-defined hooks/skills/commands/subagents/plugins/MCP servers when running against untrusted checkouts**

B Let the authoring session also perform and approve its own review to save pipeline time

C Disable CI automation entirely and require fully manual review of every PR, including untrusted ones

D Grant the CI service account full admin credentials so any required action can run without friction

CORRECT ANSWER **A**

WHY An independent headless review step avoids the self-review blind-spot problem, and bare mode specifically closes the risk of an untrusted repository's own hooks, skills, or MCP servers auto-loading and executing during a non-interactive CI run — these are two distinct, composable safeguards for two distinct risks. Letting the author review itself reintroduces blind spots, fully manual review forfeits the automation benefit, and broad admin credentials violate least-privilege regardless of the other concerns.